

Task 3: Grounded Generation & Citation



Overview

In this task, your objective is to implement, test, and validate grounded generation and structured citation mechanisms using the Jupyter notebook:

- `Task3_Grounded_Generation.ipynb`

While previous tasks established document ingestion and optimized retrieval, Task 3 focuses on **constraining the generative language model**. The goal is to enforce strict grounding so that every generated clinical statement is directly traceable to retrieved guideline evidence, with structured JSON schema validation and reliable refusal behavior for out-of-scope questions.



Objectives

By completing this task, you will:

1. **Design a Grounding System Prompt:** Construct a prompt that enforces strict context boundaries, defines output format, forbids external medical hallucinations, and mandates verifiable citations.
 2. **Enforce JSON Schema Validation:** Validate generated outputs against a formal response schema, ensuring that claims cannot have high confidence without supporting evidence and citations.
 3. **Implement Grounded Generation:** Build a pipeline that retrieves relevant context, dynamically constructs prompts, and generates structured clinical responses (supporting both live LLM calls and simulation mode).
 4. **Implement Refusal Mechanisms:** Build an automated refusal pipeline that detects low retrieval confidence or out-of-scope clinical questions and returns structured, schema-compliant refusals rather than fabricated answers.
-



Step-by-Step Task Guide

1. Setup & Environment

- Open `Task3_Grounded_Generation.ipynb`.

- Run the setup cell to rebuild/load the index and prepare retrieval components.

2. Formulate Grounding System Prompt

- Inspect the 4 core pillars of the clinical grounding prompt:
 - **Role Specification:** Acting as a citation-bound assistant rather than an unconstrained medical advisor.
 - **Context Boundary:** Restricting answers strictly to retrieved guideline chunks.
 - **Output Structure:** Requiring structured JSON with `recommendation`, `evidence`, `citations`, and `confidence`.
 - **Escape Hatch:** Explicit rules on when and how to refuse when evidence is insufficient.

3. Validate Output Schema

- Test valid clinical responses against the JSON schema to verify compliant structure.
- Execute validation on malformed/hallucinated answers (e.g., high confidence with empty evidence) to ensure the validator catches and rejects them.

4. Build Grounded Generation Function

- Assemble retrieved chunks into prompt context with document names and page numbers.
- Execute generation across sample clinical questions and validate that the output JSON adheres strictly to schema specifications.

5. Implement Refusal for Out-of-Scope Questions

- Implement confidence threshold checks in the generation pipeline.
- Test the refusal flow with out-of-scope medical questions to verify that the system issues a schema-compliant refusal without guessing.



Acceptance Criteria (Definition of Done)

- ☐ All cells in `Task3_Grounded_Generation.ipynb` execute sequentially from top to bottom without errors.
- ☐ Grounding system prompt contains all 4 necessary constraints (role, context boundary, JSON output format, escape hatch).
- ☐ Schema validation correctly passes well-formed answers and rejects unsupported answers.

- ☐ Grounded generation successfully formats context and generates schema-valid outputs.
- ☐ Out-of-scope queries trigger a structured, schema-valid refusal with confidence: "insufficient" rather than ungrounded generation.